LATTICE · THE COPY-AND-GO KIT

Markdown in. Boardroom out.

Everything on the next twelve slides is written in plain Markdown, in this file.

Read it in your editor and in the render side by side — they are the same thing.

This deck documents itself.

Every slide shows a component **and** the Markdown that made it.

01	**One file**	The whole deck. Nothing else to set up.
02	**One folder**	Everything it needs sits beside it.
03	**One line**	Picks the layout. Sixty-one to choose from.

Three steps to a rendered deck.

STEP 01

Copy the folder

Drop these files anywhere. Keep them together — paths are relative.

STEP 02

Open it in VS Code

With the Marp extension installed, the preview pane opens on this file.

STEP 03

Edit and watch

Change a heading. The preview follows. That is the whole loop.

HOW A SLIDE IS WRITTEN

One comment picks the layout.

Everything else stays Markdown.

Copy this folder. Edit this file.

A comment names the layout

`<!-- _class: kpi -->` and the next slide is a KPI row.

Headings carry the argument

Write the heading as a sentence, not a label. It is the slide's claim.

Lists become structure

A nested title-and-body pair becomes a card, a step, or a tile.

Seven keys of front matter, then one comment per slide.

```yaml
marp: true          # activates the Marp extension / marp-cli
theme: cuoio        # the palette; cuoio is the default
paginate: true      # page numbers
size: hd            # 16:9 at 1280x720
class: dark         # optional — flips the whole deck to dark
header: "Lattice · Sample Deck"
footer: "Copy this folder. Edit this file."
```

`class:` is Marp's own key: whatever you put there lands on every slide, so `class: dark` reaches Lattice's `dark` styling with nothing else to configure.

Then each slide opens with one comment naming its layout — `_class: kpi`, `_class: diagram`, `_class: quote`. That comment is the entire API.

What the folder costs you.

1

folder to copy

KEEP IT TOGETHER COMPLETE

5 MB

on disk, minified

CSS + RUNTIME + FONTS

61

layouts available

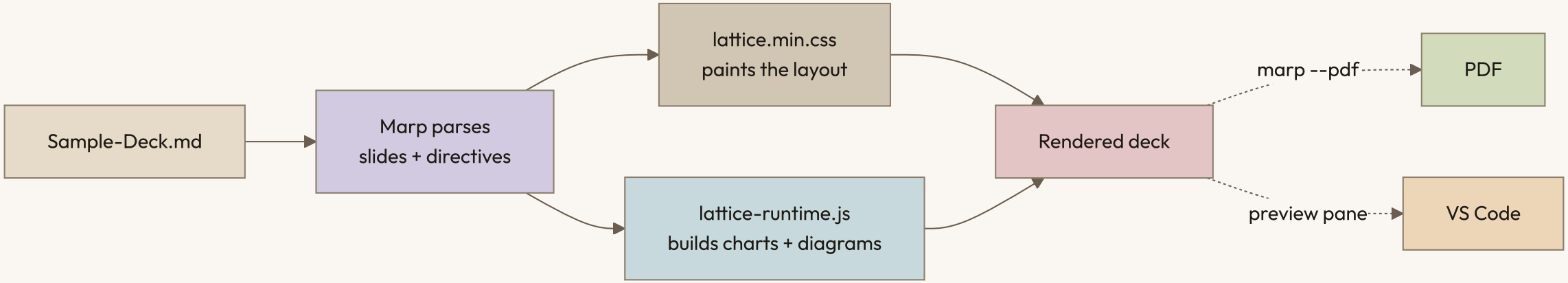
NONE TO INSTALL INCLUDED

0

build steps

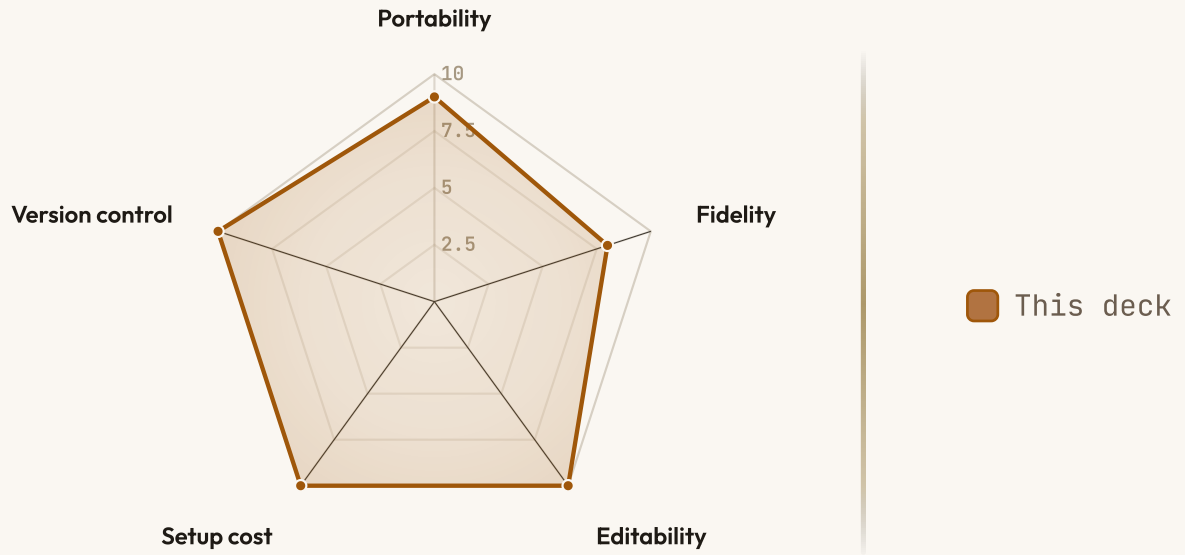
BEFORE FIRST RENDER NONE

From this file to a rendered slide.



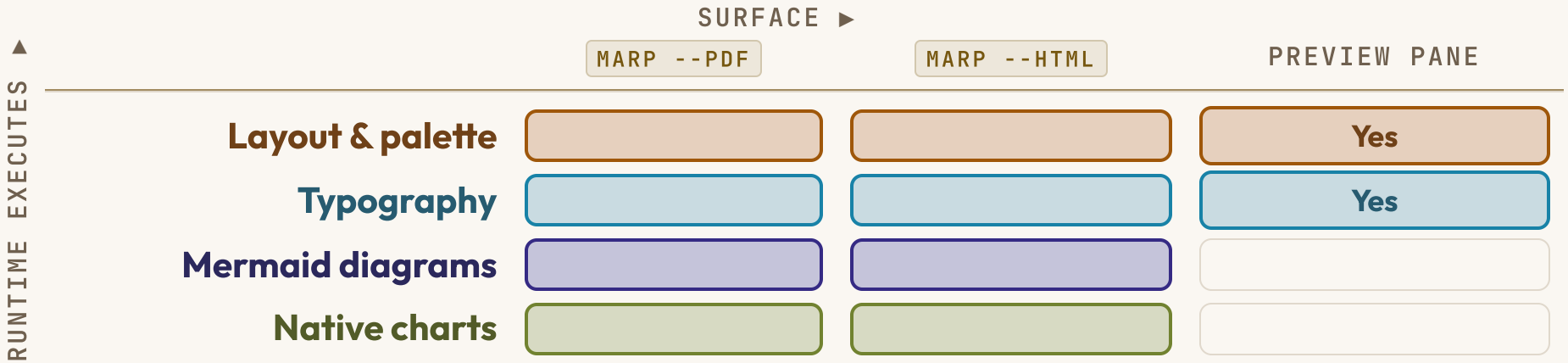
SCORED 0–10

Charts are Markdown too — no image, no plugin.



What renders where — read this before you trust a surface.

Both marp-cli routes drive a real browser, so the runtime runs and everything below is filled. The preview pane's two empty cells are unconfirmed, not broken — open this deck in VS Code and you will know.



TYPESET BY WHICHEVER ENGINE YOUR TOOL USES

Equations are inline in the Markdown.

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

WHERE

- z — the raw score vector, length K
- $\sigma(z)_i$ — probability assigned to class i
- $\sum_j e^{z_j}$ — the normalizer; outputs sum to 1
- Written as `$$ … $$` between blank lines

The parts you will touch, and what each one does.

01	This deck. Your starting point — edit it in place.	SAMPLE-DECK.MD
02	The engine. Every layout and token lives here.	LATTICE.MIN.CSS
03	The palette. A dark one ships beside it.	CUOIO.MIN.CSS
04	Builds charts and diagrams in the browser.	LATTICE-RUNTIME.MIN.JS
05	Third party. Required for diagram slides.	MERMAID-V11.MIN.JS
06	Thirty-seven files. Drop them and type falls back.	FONTS/

The config files and `NOTICE.md` are listed in the README.

"Write the deck in Markdown, render it through a real browser."

That is the one idea Lattice keeps from Marp. Everything else — the layouts, the tokens, the charts — is what gets added on top of it.

Now change something.

Edit this file · the preview follows

Swap `theme: cuoio` for another palette. Delete a slide. Add your own. Nothing here needs a build step to try.